

THE INSTRUCTIONS FILE

The single page that stops an AI asking you a question every ninety seconds — and stops your project quietly drifting away from what you asked for.

The complaint every teacher has about building with AI is not that the code is wrong. It is that you wanted to make something for your class, and instead you spent forty minutes answering questions about shades of blue.

This fixes that, and it takes about fifteen minutes once.

What an instructions file is

One short document you write before you build anything, and paste at the start of every chat about that project. It says what the thing is, what it must never do, and — the part almost nobody includes — **what the AI should do when it does not know what you want.**

Four pages, one idea, no coding

This works whether you are building a classroom game, a worksheet generator, a tracker, or nothing at all. It is a way of giving instructions, and it transfers to anything.

Where this comes from

I build software for my own high school shop course and run it with students all year. The example on page 3 is the real file from one of those projects, with nothing cleaned up for publication.

THE PROBLEM

Two ways an AI wastes your evening

Both are avoidable, and both come from the same missing page.

One: it asks instead of deciding

"Would you like the buttons rounded or square?" "Should the timer count up or down?" "Do you want this centered?"

You do not care. You have no opinion, you have never had an opinion, and being asked forces you to invent one. Ten of those and the thing you were excited to build has turned into paperwork.

Two: it drifts

Round one is exactly what you wanted. By round six it has quietly added a login screen, a settings page and a place to type student names — because that is what most software looks like, and nobody told it otherwise.

Drift is not the model being careless. It is the model filling silence with the most common answer. The fix is to stop being silent about the things you actually care about.

Both are the same missing thing

You are holding the whole project in your head, and the AI can only see this one message. **The instructions file is your head, written down,** and it is the cheapest thing you will ever do for a build.

What goes in it – a real one, not a template

This is a genuine instructions file from one of my own classroom projects. Five short sections, and none of them is about code.

1. What this is, in one sentence

A [kind of thing] for [who uses it]. Read [the other doc] before doing anything. That doc is the source of truth.

One sentence. If you cannot write it, that is worth knowing before you start building.

2. Non-negotiables – the things it may never do

- Never use the words battle, fight or enemy in anything a player reads.
- No student names anywhere, in code or on screen.
- Every question at real high school level, with a hint.

This section is the whole file. Write the rules that would make you say "no, not like that" — before you have to say it.

3. The stack, and why

Plain JavaScript, no framework, no build step. Serve it with any static server. It must run on Chromebooks and phones. Touch controls are required, not optional.

"No framework, no build step" is the line that keeps you out of a terminal. Worth writing every time.

4. Where things live, and 5. how to behave

A short map of the files, so a new chat does not reorganize your project. Then the working rules:

- Do not start the next stage until the last one works.
- Keep PROGRESS.md updated at the end of every session.
- Short, warm, dry writing. No em dashes anywhere.

The one sentence that saves the evening

If you put nothing else in the file, put this.

"When a design question is genuinely open, make the obvious choice and write it down, rather than stopping to ask."

What it does

It gives the AI permission to decide the hundred small things you have no opinion about — and it makes every one of those decisions **visible and reversible**, because it has to be written down. You get the speed of not being asked without losing the ability to say "actually, not that."

The half that makes it safe

"Write it down" means a running notes file. Mine is one page, updated at the end of every session:

```
DONE  the map screen, three routes, keyboard controls
NEXT  camps and gear
OPEN  chose three roads not four, felt cluttered
KNOWN the sound is too quiet on Chromebooks
```

That file is how you pick a project back up after a week of teaching without rereading everything. It is also how you catch a decision you disagree with, one line at a time, instead of discovering it in round twelve.

Where the line does not apply

A genuinely open question is a shade of blue. A question about what your students see, what gets stored about them, or what the thing is for is not open — and a good non-negotiables section means it never has to be asked.

YOUR TURN

Fifteen minutes, and you have one

Fill this in for whatever you are building next. Paste it at the top of every chat about that project.

What this is, in one sentence

It must never

The three things that would make you say "no, not like that."

The stack

Start with: **one HTML file, no frameworks, no build step, runs by opening it in a browser** — unless you already know you need otherwise.

And the line, copied as-is

"When a design question is genuinely open, make the obvious choice and write it down in PROGRESS.md, rather than stopping to ask."

If you want to see one doing real work

Every prompt in **Vibe Code Your Own Game: The Long Road** is written this way — twelve prompts that each spend a clause forbidding something, which is why they produce a game instead of a quiz. It is in this store, with the finished game included.

The thing worth remembering. Most of what makes AI useful is not knowing the right words to ask for. **It is deciding in advance what you will not accept**, and saying so once instead of twelve times.

If this was useful, leaving feedback costs nothing and earns you TPT credit toward anything in the store. It also tells me what to make more of.